

CHALLENGES & SOLUTIONS REPORT

VAGDEVI MALINENI

Project Name

Raritone – AI-Powered Virtual Try-On Platform

1. API Security

Challenge

Ensuring that sensitive APIs could only be accessed by authenticated and authorized users. Without proper security, unauthorized users could access protected resources, modify data, or perform malicious activities.

Solution

- Implemented JWT Authentication for secure user login.
- Added Role-Based Authorization for admin and user access control.
- Protected private routes using authentication middleware.
- Applied Rate Limiting to prevent brute-force attacks.
- Used Helmet middleware to secure HTTP headers.

Result

- ✓ Secure API access
- ✓ Protected sensitive endpoints
- ✓ Improved application security

2. Database Design

Challenge

Designing a database structure capable of handling users, products, avatars, measurements, try-on results, and future AI-generated data without redundancy.

Solution

- Designed separate MongoDB collections for each module.
- Established relationships using ObjectId references.
- Followed normalization principles to reduce duplication.
- Planned scalable schema structures for future expansion.

Result

- ✓ Organized database architecture
- ✓ Better maintainability
- ✓ Easy future scalability

3. Image Upload Handling

Challenge

Managing large image uploads while maintaining application performance and storage efficiency.

Solution

- Used Multer for file upload handling.
- Implemented Sharp for image compression and resizing.
- Integrated Cloudinary for cloud storage.
- Leveraged CDN delivery for optimized image access.
- Added cleanup mechanisms to remove temporary files.

Result

- ✓ Reduced image size
- ✓ Faster upload and delivery

✓ Lower storage costs

4. Team Coordination

Challenge

Multiple team members worked on different modules simultaneously, creating challenges in communication, task tracking, and integration.

Solution

- Divided work into clearly defined modules.
- Used GitHub for version control and collaboration.
- Conducted regular progress updates.
- Maintained shared documentation for consistency.
- Coordinated API and database design discussions.

Result

- ✓ Improved collaboration
- ✓ Reduced development conflicts
- ✓ Better project management

5. AI Integration Planning

Challenge

The AI-powered virtual try-on system required planning for future integration despite AI model development being outside the initial backend implementation phase.

Solution

- Designed AI-ready database schemas.
- Created TryOn and ProductMapping collections.
- Planned AI workflow architecture.
- Defined input and output structures for AI services.

- Researched body analysis and size recommendation approaches.

Result

- ✓ AI-ready backend architecture
- ✓ Easier future AI integration
- ✓ Structured virtual try-on workflow

6. Authentication & Authorization

Challenge

Managing user sessions securely while maintaining scalability and performance.

Solution

- Implemented JWT token-based authentication.
- Created middleware for token verification.
- Added role-based permissions.
- Protected sensitive APIs and admin routes.

Result

- ✓ Secure user management
- ✓ Scalable authentication mechanism
- ✓ Better access control

7. Request Validation

Challenge

Preventing invalid or incomplete data from entering the system and causing application errors.

Solution

- Implemented Joi validation schemas.

- Added request validation middleware.
- Validated required fields and data types.
- Returned meaningful validation error messages.

Result

- ✓ Improved data quality
- ✓ Reduced runtime errors
- ✓ Better API reliability

8. Cloud Storage Integration

Challenge

Storing and managing media files efficiently without overloading the backend server.

Solution

- Integrated Cloudinary cloud storage.
- Stored only image URLs in MongoDB.
- Used Cloudinary transformations for optimization.
- Utilized CDN delivery for faster access.

Result

- ✓ Reduced server load
- ✓ Faster image delivery
- ✓ Improved scalability

9. Scalability Planning

Challenge

Ensuring the backend can handle increased users, products, and AI processing requirements in the future.

Solution

- Researched Redis caching.
- Studied load balancing strategies.
- Planned microservices architecture.
- Designed cloud-ready deployment architecture.
- Evaluated CDN optimization techniques.

Result

- ✓ Future-ready architecture
- ✓ Improved scalability planning
- ✓ Better performance preparation

10. Documentation Management

Challenge

Creating professional and comprehensive project documentation covering architecture, APIs, security, database design, and deployment strategies.

Solution

- Prepared structured technical documentation.
- Documented APIs and database schemas.
- Created architecture and workflow diagrams.
- Compiled deployment and monitoring research.
- Maintained consistent reporting formats.

Result

- ✓ Complete project documentation
- ✓ Easier knowledge transfer
- ✓ Professional project presentation

Conclusion

Throughout the development of the Raritone project, several technical and organizational challenges were encountered, including API security, database design, media management, team collaboration, and AI integration planning. By implementing modern technologies, following best practices, and maintaining effective collaboration, these challenges were successfully addressed. The solutions developed not only resolved current issues but also prepared the platform for future scalability, cloud deployment, and AI-powered enhancements.